JAVA INTERVIEW PREPARATION

CHAPTER 45

DISTRIBUTED CONSISTENCY:
OUTBOX, IDEMPOTENCY, AND SAGAS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Distributed Consistency: Idempotency, Transactional Outbox, Sagas, and Reconciliation

Senior / Lead Java Developer Reading Chapter

A database transaction gives atomicity within one transactional resource. A business workflow crossing services, databases, brokers, payment providers, or email systems has no automatic global rollback. Distributed consistency therefore means designing intermediate states, durable intent, idempotent actions, compensation, and repair so the business invariant eventually reaches an acceptable outcome.

The senior question is not "How do I make all services ACID?" It is "Which invariant must be immediate, which can converge, who owns each transition, and how does the system recover from every interruption?"

Local Atomicity Before Distributed Workflow

Keep the strongest invariant within one service and one local transaction whenever possible. An Order service can atomically create an order and an outbox record because both live in its database. It cannot atomically commit that database and an independent Payment service by ordinary `@Transactional` annotation.

```
Order database transaction:
  insert order(status = PENDING_PAYMENT)
  insert outbox(event = OrderPlaced)
  COMMIT both or neither

later:
  outbox relay -> broker -> Payment service
```

If a workflow constantly needs several services to approve one basic state transition synchronously and atomically, revisit the boundary. Distribution should not split a tightly coupled invariant merely to match an architecture diagram.

Why Dual Writes Fail

The naive sequence updates a database and publishes a message as two independent operations.

```
Option A: database commit -> publish
  crash after commit: state changed, message lost

Option B: publish -> database commit
  database fails: message describes state that never committed
```

Retries do not remove the uncertainty. Retrying publication can duplicate an event; retrying the database operation can duplicate a business effect. Two-phase commit can coordinate supported resources, but increases coupling, latency, and failure complexity and does not include many external systems. Most microservice workflows prefer local transactions plus durable asynchronous coordination.

Transactional Outbox

The outbox pattern records the business change and publication intent in the same local database transaction. A separate relay publishes committed outbox rows.

```
begin;
update orders set status = 'CONFIRMED', version = version + 1
 where id = :id and version = :expected;
insert into outbox_event(
    event_id, aggregate_type, aggregate_id, event_type, payload, occurred_at)
values (:eventId, 'Order', :id, 'OrderConfirmed', :payload, :occurredAt);
commit;
```

The relay can poll unpublished rows or use change-data capture. Polling is simpler and application-controlled but adds query load and delay. Change-data capture follows the database log and can reduce polling, but adds connector infrastructure and schema/operation expertise.

The relay marks or removes a row only after the broker acknowledges publication. A crash after broker acknowledgement but before marking the row republishes the event. Therefore the outbox prevents loss, not duplicates; consumers still need idempotency.

Outbox Schema and Relay Ownership

Use a stable event ID, aggregate identity, event type, schema version, occurrence time, payload, publication state, attempt count, and diagnostic timestamps. Avoid storing a live entity reference or rebuilding the event later from current state, because the entity may have changed before publication.

```
outbox row is an immutable publication fact
event payload reflects transaction-time state
relay metadata tracks delivery attempts separately
```

Ordering is usually required per aggregate, not globally. Partition published events by aggregate ID and preserve a sequence or version. Several relay workers can claim rows with database locking or leases, but must avoid concurrently reordering events for the same aggregate.

Backpressure applies to the relay. If the broker is down, the outbox grows. Monitor oldest unpublished age, row count, attempts, storage, and publication throughput. Retention and cleanup must not delete evidence needed for recovery or audit.

Treat relay publication as privileged infrastructure. Payloads can contain sensitive data; restrict database and broker access, encrypt transport, and redact diagnostics. A support action that republishes rows is a production mutation requiring audit.

Inbox and Idempotent Consumers

The receiving side can use an inbox or processed-message table. The message claim and local state change share one transaction.

```
begin;
insert into processed_message(consumer, message_id)
values ('shipping-service', :messageId)
on conflict do nothing;

-- only when the insert claimed the message
update shipment ...;
commit;
```

The unique key closes concurrent duplicate races. Store enough information to diagnose processing without retaining unnecessary personal payloads. Retention must exceed any replay window that can reintroduce the message.

Natural idempotency can complement the inbox: conditional status changes, unique external reference, and version checks. Do not rely on a `SELECT` followed by `INSERT` without a constraint; two consumer threads can

both observe absence.

Idempotency for External Side Effects

An inbox transaction cannot roll back an email already sent or a payment already authorized. Use the external provider's idempotency mechanism when available and persist the provider operation key before calling.

```
workflow step ID -> stable provider idempotency key
attempt 1 times out after provider commit
attempt 2 uses same key -> provider returns original result
```

If the provider has no idempotency support, query operation status by a stable business reference before repeating, or introduce a human reconciliation state when truth remains uncertain. Pretending timeout equals failure risks duplicate financial effects.

Separate attempt identity from business-operation identity. Every HTTP attempt can have a new trace span, while all attempts for one payment use the same idempotency key.

Saga as a Sequence of Local Transactions

A saga coordinates local transactions across services. Each successful step advances the workflow; a later failure may trigger compensating actions.

```
Create Order
  -> Reserve Inventory
  -> Authorize Payment
  -> Arrange Shipment

failure at shipment:
  -> Void Payment Authorization
  -> Release Inventory
  -> Mark Order REJECTED
```

Compensation is not database rollback. The world may have observed intermediate facts, prices may change, inventory may be consumed, and a refund is a new financial transaction rather than erasing history. Compensation must be idempotent, auditable, and allowed to fail and retry.

Some actions are irreversible. Sending a package, issuing a legal document, or notifying a customer may require the workflow to delay the irreversible pivot until earlier uncertainty is resolved. Saga design should identify pivot and compensatable steps explicitly.

Orchestration Versus Choreography

In orchestration, one workflow component sends commands, records state, waits for replies, applies timeouts, and decides compensation.

```
Order Saga
  -> ReserveInventory command
  <- InventoryReserved event
  -> AuthorizePayment command
  <- PaymentDeclined event
  -> ReleaseInventory command
```

Orchestration makes current state, timeout ownership, and recovery visible. The orchestrator can become complex or overly centralized if it begins owning domain decisions that belong to participants.

In choreography, services react to events without a central controller. It can work well for simple fan-out and loosely coupled reactions. Long chains become difficult to understand: hidden cycles, unclear completion, and many services reacting to the same internal event create an implicit distributed workflow.

Choose based on workflow complexity and ownership, not ideology. A common design uses orchestration for the core business transaction and choreography for optional reactions such as analytics or notifications.

Durable Saga State Machine

Saga state must survive restart. Store workflow ID, business identity, current state, completed steps, deadlines, message identities, attempt metadata, and failure reason in durable storage.

```
PENDING_PAYMENT
  payment accepted -> PENDING_INVENTORY
  payment declined -> REJECTED
  deadline expired -> COMPENSATING

COMPENSATING
  payment voided and inventory released -> CANCELLED
  repeated failure -> MANUAL_REVIEW
```

Transition with optimistic locking or a conditional update so duplicate and concurrent replies cannot advance the same workflow twice. Validate that an incoming reply is expected in the current state. Late success after timeout may require compensation rather than normal advancement.

Timers are messages too. Persist deadlines and schedule or scan them durably. An in-memory scheduled task disappears on restart and can run on every replica. Timeout processing must be idempotent and race safely with a success response arriving at the same moment.

Eventual Consistency as a Product Contract

Eventual consistency does not mean "the system will probably catch up." Define the convergence path, expected delay, conflict rule, and user-visible intermediate state.

A newly confirmed order might appear immediately in the owning service but several seconds later in a search view. The UI can display `PROCESSING`, use read-your-write routing to the owner, or poll an operation resource. Returning 404 from a lagging projection immediately after successful creation produces a confusing contract.

Staleness budgets should be measured. Observe event age and projection lag against the user promise. If a projection falls behind retention, rebuild it from a snapshot plus log or from the source of truth through a controlled backfill.

Reservations and Try-Confirm-Cancel

Some scarce resources need an explicit temporary reservation. Try-confirm-cancel separates the workflow into a provisional hold, a final confirmation, and a cancellation or expiry.

```
TRY:     reserve 2 inventory units until deadline D
CONFIRM: convert reservation to committed allocation
CANCEL:  release reservation, or let it expire safely
```

Every phase must be idempotent and tied to one reservation identity. Confirmation after expiry must not silently succeed against stock now allocated elsewhere. Cancellation racing confirmation needs a conditional state transition. A background expiry process is durable, retryable, and observable rather than an in-memory timer.

This pattern improves contention control but temporarily reduces available capacity and adds cleanup. It is useful when the business can represent a hold honestly. It is not a generic replacement for transactions, and long reservations can become denial-of-inventory attacks unless identity, limits, and expiry are enforced.

Observability of Business Convergence

Technical delivery metrics are necessary but insufficient. A broker can have zero lag while thousands of orders remain stuck in `COMPENSATING`. Measure workflow state counts, age in each non-terminal state, deadline misses, compensation attempts, manual-review backlog, outbox age, inbox duplicates, and reconciliation mismatches.

Use stable workflow and failure categories as metric labels, not workflow IDs. Traces show one saga's command and event chain, while durable workflow history remains the authoritative operational timeline when sampling omits spans.

Alerts should follow business impact. One retry is normal; a rising oldest-pending age or growing manual-review queue threatens the service promise. Runbooks must identify safe replay, compensation, and reconciliation commands and state which evidence to preserve before modifying workflow state.

Reconciliation Is a Primary Mechanism

Even well-designed messaging and sagas encounter operational defects, bad deployments, expired credentials, and provider ambiguity. Reconciliation compares independent records and repairs divergence.

```
local payment state       provider payment state
AUTHORIZED                AUTHORIZED       -> consistent
PENDING for 30 minutes    AUTHORIZED       -> recover success
AUTHORIZED                NOT_FOUND        -> investigate or compensate
```

Reconciliation can be scheduled, event-triggered, or initiated by support. It needs stable external references, bounded query rates, idempotent repair actions, and audit. Metrics should report age and count of unresolved states, not only exceptions.

Manual review is a valid terminal operational state when automatic action could cause greater harm. Give operators the evidence and safe commands needed to resolve it; do not leave them editing database rows directly.

Isolation and Concurrency Inside Each Service

Eventual consistency between services does not remove local concurrency control. Use unique constraints, optimistic versions, conditional updates, and appropriate database isolation to protect each service's invariants.

Two saga messages can race: a cancellation and a successful payment reply. A state-machine conditional update decides which transition is legal. Relying only on handler execution order fails under multiple consumers, retries, and rebalancing.

Avoid holding local database transactions open while waiting for remote work. Persist intent, commit, communicate, then process the reply in another local transaction. This releases connections and locks and makes the wait durable.

Testing Distributed Consistency

Test every crash boundary:

- After business commit but before outbox publication.
- After broker publication but before relay acknowledgement state.
- After consumer effect but before offset acknowledgement.
- During compensation after some compensations succeeded.

- When a late reply races a timeout.
- When the same command or event arrives concurrently.
- When the provider committed an operation but the response was lost.

Property-based state-machine tests can generate valid and invalid transition sequences. Integration tests need the real database constraints and broker behavior. End-to-end tests should inspect durable state and messages, not merely wait for a final HTTP response.

Operational drills should pause the broker, stop relays, introduce poison records, expire provider credentials, and restart workers during processing. Verify lag, alerting, recovery, and reconciliation without duplicate effects.

Common Senior-Level Traps

The first trap is dual-writing to a database and broker and assuming retries make it atomic. The second is calling compensation rollback. The third is treating eventual consistency as an excuse for undefined user behavior.

Other traps include deleting outbox or inbox records before the replay window closes, rebuilding an event from mutable current state, waiting on remote services inside a database transaction, in-memory saga timers, choreography with no owner, and dashboards that show exceptions but not unresolved business states.

A Strong Interview Answer

> **I keep immediate invariants inside one service and use local transactions for each durable step. To avoid database-message dual writes, the business change and immutable outbox event commit together; a relay publishes at least once, and consumers atomically deduplicate message identity with their local effect. A saga models the cross-service workflow as durable state transitions with explicit deadlines, idempotent commands, compensations, and manual-review states. Compensation is a new business action, not rollback. I choose orchestration when timeout and recovery ownership must be visible, use choreography for loosely coupled reactions, and treat reconciliation, staleness metrics, and crash-boundary tests as primary correctness mechanisms.**

Interview-Ready Summary

Distributed consistency is built from local atomicity, durable intent, repeatable effects, explicit workflow states, and repair. Transactional outbox prevents lost publication but not duplicates. Inbox or natural idempotency makes repeated consumption safe.

Sagas coordinate local transactions and compensation without pretending to provide global rollback. Eventual consistency needs a product-visible convergence contract. Reconciliation closes gaps that normal message flow and retry cannot safely resolve.

Revision Notes

- Keep strongly consistent invariants within one transactional owner.
- Spring transactions do not span arbitrary services and resources.
- Database-plus-broker dual writes have an unavoidable failure window.
- Outbox state and business state commit in one local transaction.
- Polling and change-data capture are alternative relay mechanisms.
- Relay acknowledgement failure can republish; consumers need idempotency.
- Store immutable transaction-time event data in the outbox.
- Preserve required ordering per aggregate rather than globally.
- Monitor oldest unpublished event age and outbox growth.
- Inbox claim and local business effect must be atomic.
- Back duplicate checks with a database uniqueness constraint.
- External side effects need stable provider idempotency keys.
- A timeout can leave an external operation outcome unknown.
- A saga is a sequence of durable local transactions.
- Compensation is a new auditable action, not historical erasure.
- Identify compensatable, pivot, and irreversible steps.
- Orchestration makes workflow state and recovery ownership explicit.
- Choreography is best for simple loosely coupled reactions.
- Persist saga state, deadlines, replies, and transition versions.
- Race late success against timeout through conditional state transitions.
- Eventual consistency needs a defined staleness and user contract.
- Reconciliation detects and repairs divergence independently.
- Test every commit, publish, acknowledge, timeout, and compensation boundary.